

Graphic Rants

Graphical, textual ramblings of a coder.

Saturday, August 3, 2013

Specular BRDF Reference

While I worked on our [new shading model for UE4](#) I tried many different options for our specular BRDF. Specifically, I tried many different terms for to Cook-Torrance microfacet specular BRDF:

$$f(\mathbf{l}, \mathbf{v}) = \frac{D(\mathbf{h})F(\mathbf{v}, \mathbf{h})G(\mathbf{l}, \mathbf{v}, \mathbf{h})}{4(\mathbf{n} \cdot \mathbf{l})(\mathbf{n} \cdot \mathbf{v})}$$

Directly comparing different terms requires being able to swap them while still using the same input parameters. I thought it might be a useful reference to put these all in one place using the same symbols and same inputs. I will use the same form as Naty [1], so please look there for background and theory. I'd like to keep this as a living reference so if you have useful additions or suggestions let me know.

First let me define alpha that will be used for all following equations using UE4's roughness:

$$\alpha = roughness^2$$

Normal Distribution Function (NDF)

The NDF, also known as the specular distribution, describes the distribution of microfacets for the surface. It is normalized [12] such that:

$$\int_{\Omega} D(\mathbf{m})(\mathbf{n} \cdot \mathbf{m}) d\omega_i = 1$$

It is interesting to notice all models have $\frac{1}{\pi\alpha^2}$ for the normalization factor in the isotropic case.

Blinn-Phong [2]:

$$D_{Blinn}(\mathbf{m}) = \frac{1}{\pi\alpha^2} (\mathbf{n} \cdot \mathbf{m})^{\left(\frac{2}{\alpha^2} - 2\right)}$$

This is not the common form but follows when $power = \frac{2}{\alpha^2} - 2$.

Beckmann [3]:

$$D_{Beckmann}(\mathbf{m}) = \frac{1}{\pi\alpha^2(\mathbf{n} \cdot \mathbf{m})^4} \exp\left(-\frac{(\mathbf{n} \cdot \mathbf{m})^2 - 1}{\alpha^2(\mathbf{n} \cdot \mathbf{m})^2}\right)$$

GGX (Trowbridge-Reitz) [4]:

$$D_{GGX}(\mathbf{m}) = \frac{\alpha^2}{\pi((\mathbf{n} \cdot \mathbf{m})^2(\alpha^2 - 1) + 1)^2}$$

GGX Anisotropic [5]:

$$D_{GGX_{aniso}}(\mathbf{m}) = \frac{1}{\pi\alpha_x\alpha_y} \frac{1}{\left(\frac{(\mathbf{x} \cdot \mathbf{m})^2}{\alpha_x^2} + \frac{(\mathbf{y} \cdot \mathbf{m})^2}{\alpha_y^2} + (\mathbf{n} \cdot \mathbf{m})^2\right)^2}$$

Geometric Shadowing

The geometric shadowing term describes the shadowing from the microfacets. This means ideally it should depend on roughness and the microfacet distribution.

Implicit [1]:

$$G_{Implicit}(\mathbf{l}, \mathbf{v}, \mathbf{h}) = (\mathbf{n} \cdot \mathbf{l})(\mathbf{n} \cdot \mathbf{v})$$

Neumann [6]:

About Me

Brian Kari

I am a Engineer at Epic Games. This blog is a dump game art thoughts and some graphics concepts.

[View my portfolio](#)

Blog Archive

- ▼ 2026 (5)
 - ▼ March (1)
 - Variable :
- ▼ February
 - Nanite +
 - How to te
 - Possible tessell
 - Nanite Te

- 2018 (4)
- 2014 (1)
- ▼ 2013 (3)
 - Decemb
 - ▼ August (
 - Specular
 - July (1)

- 2012 (2)
- 2011 (2)
- 2010 (1)
- 2009 (6)
- 2008 (8)

$$G_{Neumann}(\mathbf{l}, \mathbf{v}, \mathbf{h}) = \frac{(\mathbf{n} \cdot \mathbf{l})(\mathbf{n} \cdot \mathbf{v})}{\max(\mathbf{n} \cdot \mathbf{l}, \mathbf{n} \cdot \mathbf{v})}$$

Cook-Torrance [11]:

$$G_{Cook-Torrance}(\mathbf{l}, \mathbf{v}, \mathbf{h}) = \min \left(1, \frac{2(\mathbf{n} \cdot \mathbf{h})(\mathbf{n} \cdot \mathbf{v})}{\mathbf{v} \cdot \mathbf{h}}, \frac{2(\mathbf{n} \cdot \mathbf{h})(\mathbf{n} \cdot \mathbf{l})}{\mathbf{v} \cdot \mathbf{h}} \right)$$

Kelemen [7]:

$$G_{Kelemen}(\mathbf{l}, \mathbf{v}, \mathbf{h}) = \frac{(\mathbf{n} \cdot \mathbf{l})(\mathbf{n} \cdot \mathbf{v})}{(\mathbf{v} \cdot \mathbf{h})^2}$$

Smith

The following geometric shadowing models use Smith's method[8] for their respective NDF. Smith breaks G into two components: light and view, and uses the same equation for both:

$$G(\mathbf{l}, \mathbf{v}, \mathbf{h}) = G_1(\mathbf{l})G_1(\mathbf{v})$$

I will define G_1 below for each model and skip duplicating the above equation.

Beckmann [4]:

$$c = \frac{\mathbf{n} \cdot \mathbf{v}}{\alpha \sqrt{1 - (\mathbf{n} \cdot \mathbf{v})^2}}$$

$$G_{Beckmann}(\mathbf{v}) = \begin{cases} \frac{3.535c + 2.181c^2}{1 + 2.276c + 2.577c^2} & \text{if } c < 1.6 \\ 1 & \text{if } c \geq 1.6 \end{cases}$$

Blinn-Phong:

The Smith integral has no closed form solution for Blinn-Phong. Walter [4] suggests using the same equation as Beckmann.

GGX [4]:

$$G_{GGX}(\mathbf{v}) = \frac{2(\mathbf{n} \cdot \mathbf{v})}{(\mathbf{n} \cdot \mathbf{v}) + \sqrt{\alpha^2 + (1 - \alpha^2)(\mathbf{n} \cdot \mathbf{v})^2}}$$

This is not the common form but is a simple refactor by multiplying by $\frac{\mathbf{n} \cdot \mathbf{v}}{\mathbf{n} \cdot \mathbf{v}}$.

Schlick-Beckmann:

Schlick [9] approximated the Smith equation for Beckmann. Naty [1] warns that Schlick approximated the wrong version of Smith, so be sure to compare to the Smith version before using.

$$k = \alpha \sqrt{\frac{2}{\pi}}$$

$$G_{Schlick}(\mathbf{v}) = \frac{\mathbf{n} \cdot \mathbf{v}}{(\mathbf{n} \cdot \mathbf{v})(1 - k) + k}$$

Schlick-GGX:

For UE4, I used the Schlick approximation and matched it to the GGX Smith formulation by remapping k [10]:

$$k = \frac{\alpha}{2}$$

Fresnel

The Fresnel function describes the amount of light that reflects from a mirror surface given its index of refraction. Instead of using IOR we instead use the parameter or F_0 which is the reflectance at normal incidence.

None:

$$F_{None}(\mathbf{v}, \mathbf{h}) = F_0$$

Schlick [9]:

$$F_{Schlick}(\mathbf{v}, \mathbf{h}) = F_0 + (1 - F_0)(1 - (\mathbf{v} \cdot \mathbf{h}))^5$$

Cook-Torrance [11]:

$$\eta = \frac{1 + \sqrt{F_0}}{1 - \sqrt{F_0}}$$

$$c = \mathbf{v} \cdot \mathbf{h}$$

$$g = \sqrt{\eta^2 + c^2 - 1}$$

$$F_{Cook-Torrance}(\mathbf{v}, \mathbf{h}) = \frac{1}{2} \left(\frac{g - c}{g + c} \right)^2 \left(1 + \left(\frac{(g + c)c - 1}{(g - c)c + 1} \right)^2 \right)$$

Optimize

Be sure to optimize the BRDF shader code as a whole. I choose these forms of the equations to either match the literature or to demonstrate some property. They are not in the optimal form to compute in a pixel shader. For example, grouping Smith GGX with the BRDF denominator we have this:

$$\frac{G_{GGX}(\mathbf{l})G_{GGX}(\mathbf{v})}{4(\mathbf{n} \cdot \mathbf{l})(\mathbf{n} \cdot \mathbf{v})}$$

In optimized HLSL it looks like this:

```
float a2 = a*a;
float G_V = NoV + sqrt( (NoV - NoV * a2) * NoV + a2 );
float G_L = NoL + sqrt( (NoL - NoL * a2) * NoL + a2 );
return rcp( G_V * G_L );
```

If you are using this on an older non-scalar GPU you could vectorize it as well.

References

- [1] Hoffman 2013, "[Background: Physics and Math of Shading](#)"
- [2] Blinn 1977, "Models of light reflection for computer synthesized pictures"
- [3] Beckmann 1963, "The scattering of electromagnetic waves from rough surfaces"
- [4] Walter et al. 2007, "[Microfacet models for refraction through rough surfaces](#)"
- [5] Burley 2012, "[Physically-Based Shading at Disney](#)"
- [6] Neumann et al. 1999, "[Compact metallic reflectance models](#)"
- [7] Kelemen 2001, "[A microfacet based coupled specular-matte brdf model with importance sampling](#)"
- [8] Smith 1967, "Geometrical shadowing of a random rough surface"
- [9] Schlick 1994, "[An Inexpensive BRDF Model for Physically-Based Rendering](#)"
- [10] Karis 2013, "[Real Shading in Unreal Engine 4](#)"
- [11] Cook and Torrance 1982, "[A Reflectance Model for Computer Graphics](#)"
- [12] Reed 2013, "[How Is the NDF Really Defined?](#)"

Posted by [Brian Karis](#) at [9:16 PM](#)

14 comments:

Unknown said...



Hi,
Thanks to post all these explanations about the physical based lighting. I have tried to implement it and I got a result but I'm not sure if the calcule is really correct. Maybe could you tell me, my pixel shader code is here : <http://pastebin.com/yhkL5KmC>.
Thanks for the help

[August 23, 2013 at 9:33 PM](#)

bj said...



Thanks for posting all these, and for the great Siggraph presentation.

[September 1, 2013 at 4:54 PM](#)

Rook Bloodhoof said...



Thanks for the blog and video, it makes very interesting reading.

[September 3, 2013 at 9:09 AM](#)

Morgan said...



Can you add the max(0, ...) calls? E.g., in Blinn-Phong, the central factor should be: max(0, n · m)

[March 28, 2014 at 7:56 AM](#)

Changmin said...



Thanks for the posting. :)

I think that "m" in NDF may be half vector. Is it right?

[March 31, 2014 at 8:46 PM](#)

Brian Karis said...

Yes m stands for the microfacet normal. The NDF describes the distribution of these microfacets. For the microfacet to reflect to the view the vector needs to be half way between the light and the view vector. So, the NDF is then telling you the amount of microfacets that line up to reflect towards that view. This is a long way to say that yes in practice you input the half vector for m.

March 31, 2014 at 8:55 PM

Changmin said...



I could understand it easily. Thanks to your lucid explanation!!

March 31, 2014 at 10:31 PM

Morgan said...



For the Beckmann/Phong G1, the definition of c is missing. It is:

$$c = \frac{[m \cdot n]}{\alpha \sqrt{1 - (m \cdot n)^2}}$$

April 2, 2014 at 5:33 AM

Anonymous said...

Great reference, thanks for writing this up! Blinn's paper is available online here <http://research.microsoft.com/pubs/73852/p192-blinn.pdf> if you'd like to add it to the references.

April 16, 2014 at 8:24 AM

Brian Karis said...



Morgan,

These functions are only defined within the hemisphere. max(0,x) or saturating are ways to make undefined parameters do something sensible. Translating these into shader code will require it as well as preventing divide by zero. I changed the Kelemen function to make it clearer. That was the only one that was ambiguous.

July 1, 2014 at 9:39 PM

Brian Karis said...



Replaced the Kelemen function.

Equivalent but this one uses terms more similar to the others. Uses double angle cosine identity.

$$\text{dot}(V,L) = 2 * \text{dot}(V,H)^2 - 1$$

July 1, 2014 at 9:48 PM

Calvin Hsu said...



Hi Brian, thanks for posting this.

I realize in your course notes you mention that you decided to follow disney's model of re-mapping roughness using $(\text{Roughness}+1)/2$.

In the UE4 source it appears that the mapping has gone away even for analytic light sources, did it not make a difference?

September 19, 2014 at 6:29 PM

Brian Karis said...



It does make a difference but I removed it a while ago because I felt it is less accurate.

Disney has recently done the same, mentioned in an addendum to their course notes.

September 21, 2014 at 9:59 PM

Jeremy Ong said...



Those that implement this should note the singularity that occurs when $n \cdot v$ is approximately 0 (diverging fresnel reflectance and complete geometric attenuation competing with each other). When computing the final result, $n \cdot v$ should be floored to some epsilon to prevent artifacts. In any case, viewing a surface at a completely orthogonal angle is non-physical and should not be modeled.

March 11, 2015 at 8:50 PM

[Post a Comment](#)

[Newer Post](#)

[Home](#)

[Older Post](#)

Subscribe to: [Post Comments \(Atom\)](#)